ResolveFlow on Kubernetes/EKS: Learning Notes

Personal learning notes. Everything here is split into **what was actually run** versus **what was only designed**, so every claim stays defensible. Re-check the status table before relying on it.

1. Your 30-second pitch (honest version)

"ResolveFlow is my LangGraph agent that diagnoses GitHub issues, with a human approval gate before it writes anything. It runs on Render today. As a learning exercise I containerized it with a hardened image, ran it on Kubernetes with probes, resource limits, graceful shutdown and a zero-unavailable rolling update, rehearsed a failed rollout and rollback, and deployed it to a 2-node EKS cluster in Sydney from an ECR image pinned by digest (test environment, reached by port-forward, no production traffic). Along the way I found that concurrent resumes can post duplicate GitHub comments, and designed and tested a claim-plus-marker fix on Postgres."

2. Status table (re-verify before relying on it)

Item	Status	Say it as
Hardened image (non-root, no .env/frontend, exec-form CMD)	DONE (local)	"I built..."
Deployment: 2 replicas, startup/readiness/liveness, preStop, memory limit, maxUnavailable 0	DONE on local Kubernetes	"On a local cluster I..."
Bad rollout keeps old pods serving; rollout undo	DONE (local demo)	"I rehearsed a failed rollout and rollback"
EKS control plane in ap-southeast-2 (v1.35)	DONE (ACTIVE)	"I provisioned an EKS cluster with eksctl"
Worker nodes on EKS (2 x t3.small, 2 AZs)	DONE (verified): both nodes Ready; aws-node, kube-proxy, coredns, metrics-server Running	"I stood up a 2-node EKS cluster in Sydney"
ResolveFlow running on EKS from the ECR digest	DONE (verified): 2 replicas on 2 different nodes; running imageID equals the pushed digest; UID 10001; /api/health 200, /api/auth/me 401, forged resume 401; 9 tables created in the in-cluster Postgres (throwaway, not prod)	"I deployed it to EKS as a test environment, reached via port-forward; no public ingress, no production traffic"
Failed rollout + rollback on EKS	DONE (verified): bad tag -> new pod ImagePullBackOff, both old pods stayed Ready (2/2 available); rollout undo restored the digest	"I rehearsed a failed rollout and rollback on EKS"
Image in ECR, pinned by digest	DONE (verified in ECR): tag bfe7222, sha256:9d653f83...49c, scan-on-push, immutable tags	"Pushed to ECR and referenced by digest"
AWS vector store (S3 Vectors / Bedrock KB)	DESIGN ONLY. API verified reachable in my account (ap-southeast-2); nothing built or benchmarked	"Available; evaluation planned" - never quote latency/cost as measured
Exactly-once GitHub writes (claim + marker)	DONE and tested on Postgres (demo.py)	"I designed and tested..." (not yet wired into app)
ALB/ACM/WAF, Secrets Manager, Pod Identity, GitHub OIDC CI	DESIGNED ONLY	"Designed, not deployed"

Item	Status	Say it as
Production traffic / public ingress / real OAuth login on EKS	NO (OAuth callback is tied to the Vercel domain; EKS reached by port-forward only)	Never claim

Repo evidence: merged to main (commit 858e170): deploy/, docs/EKS_DEPLOYMENT.md, docs/AWS_VECTOR_STORE_DESIGN.md, docs/exactly-once/, and these notes. Only docs and deploy files were added; the application code and the Render/Vercel setup were not changed.

3. What ResolveFlow is (know this cold)

- Frontend: React on Vercel. vercel.json proxies /api/* to the backend so the session cookie is first-party.
- Backend: FastAPI + LangGraph on Render. Graph: fetch_evidence -> normalize -> classify -> (deterministic | AI diagnosis -> independent review) -> await_approval (interrupt) -> execute (GitHub comment + label).
- State: Postgres. LangGraph checkpoints (AsyncPostgresSaver) plus app tables: sessions, thread_owners, oauth_states, analyze_calls, feedback.
- External: OpenAI (gpt-4o-mini twice), Pinecone retrieval, GitHub REST, GitHub OAuth.
- Safety: execute re-checks approved itself and posts the stored proposed_action verbatim. Groundedness/risk checks are computed in code, not by the LLM.

Why the app is easy to run on Kubernetes

All durable state lives in Postgres, so pods are stateless: any pod can resume any thread. That is the property that makes replicas, rolling updates and rescheduling safe.

Risks I found (volunteer these; it reads as seniority)

Risk	Why it matters	Fix
Duplicate GitHub comment on concurrent/replayed resume	Checkpoint and GitHub write are two systems; no atomicity	Claim row + marker (section 6)
OAuth token stored plaintext in sessions AND inside checkpoints	Anyone with DB read gets user tokens	Encrypt at rest, keep tokens out of graph state, fetch by session
Every replica runs schema setup at boot	Concurrent DDL can race on rollout	Migration Job or pg_advisory_lock
Pinecone called at import time	Outage -> crash loop before /health	Lazy init + readiness check
/api/health does not check Postgres	One endpoint for readiness and liveness	Split: liveness = process, readiness = dependencies
Rate limit only on /analyze; sessions never expire server-side	Abuse and stale tokens	Per-IP limits, TTL on sessions

4. Deployment concepts you must explain

Rollout safety

- **RollingUpdate with maxSurge 1, maxUnavailable 0**: a new pod must be Ready before an old one is removed, so capacity never drops. Demonstrated: a bad image left both old pods serving.
- **Probes**: startup (slow boot allowance), readiness (receive traffic?), liveness (restart me?). Never put a shared-dependency check in liveness: a DB blip would restart every pod at once. Be careful even in readiness: a shared dependency can pull all pods out at once.
- **Rollback**: kubectl rollout undo works because ReplicaSets are retained and images are immutable. DB migrations must be backward compatible (expand, migrate, contract).
- **Progressive delivery**: canary/rings with automatic halt on error rate (Argo Rollouts, Flagger). A widely discussed 2024 security-vendor outage came from a bad content update pushed everywhere at once; staged rollout is the lesson.

Graceful shutdown

On termination Kubernetes removes the pod from endpoints and runs preStop, then sends SIGTERM, then waits terminationGracePeriodSeconds, then SIGKILL. Endpoint removal and SIGTERM race, so preStop sleep 5 lets load balancers drain. uvicorn graceful timeout must be shorter than the grace period. Use exec-form CMD so the app is PID 1 and actually receives SIGTERM.

Resources

- Requests drive scheduling; limits are enforced. Memory limit = OOM kill safety net.
- A **CPU limit** is enforced by CFS throttling: a quota per 100 ms window, then the container is frozen even if the node is idle. That produces latency spikes, so latency-sensitive services usually set CPU requests but no CPU limit.
- Add HPA, a PodDisruptionBudget (minAvailable 1), and topologySpreadConstraints across AZs so a node drain cannot remove all replicas.

Security (the area that gets probed hardest)

- Container: non-root, readOnlyRootFilesystem, drop all capabilities, no privilege escalation, minimal build context (no .env).
- Supply chain: deploy by **digest** (tags are mutable), scan on push, SBOM, sign with cosign, enforce at admission (Kyverno/Gatekeeper).
- Secrets: never in image/git. K8s Secrets are only base64; enable KMS envelope encryption; better Secrets Manager via CSI/External Secrets.
- Workload identity: **IRSA or EKS Pod Identity**, no long-lived keys on pods. In my account an org SCP blocked the IAM OIDC provider, so IRSA was unavailable and Pod Identity is the design.
- CI: GitHub OIDC assumes an IAM role (trust policy scoped to repo and branch). No stored AWS keys.
- Network: private subnets for nodes, default-deny NetworkPolicy, TLS at ALB (ACM), WAF in front. Separate accounts/namespaces per environment.

5. Debugging stories (STAR) - these prove you actually did it

Story	What happened	What you did / learned
Region-locked SCP	EKS/ECR/EC2 returned explicit deny in us-east-1	Read the error (it named a service control policy), probed regions, found ap-southeast-2 allowed, moved there. Did not bypass the guardrail.
OIDC SCP	eksctl failed at 'associate IAM OIDC provider' (second SCP)	Removed the OIDC setting, created add-ons directly via the EKS API. Design uses Pod Identity instead of IRSA.
Nodegroup stuck 21 min	health.issues was empty, so no clue there	Walked down: CloudFormation events -> EC2 (no instances) -> ASG scaling activities: 'instance type not eligible for Free Tier'. Switched to t3.small.
My own script bug	ECR script built the Sydney registry URL but aws calls used default us-east-1	Made region one explicit exported input. Lesson: ambient defaults put things in the wrong region.
Lost evidence	I recycled crash-looping pods before reading logs	Rule: kubectl logs --previous and describe pod BEFORE deleting anything.

Debug path for a stuck rollout

```
kubectl rollout status deploy/X
kubectl describe pod <pod>      # Events: probe failures, ImagePull, Insufficient cpu/memory, Too many pods
kubectl logs <pod> --previous   # why it crashed
kubectl get events --sort-by=.lastTimestamp
# decide: fix forward or kubectl rollout undo
```

6. Exactly-once GitHub writes and the Postgres 'lock'

The problem

Approve does two things in two systems: post a GitHub comment, then record done in the database. Two bugs: (A) two requests at once both see 'not posted' and both post; (B) the pod posts, crashes before recording, and the retry posts again.

The fix, in three parts

- **Claim**: a side_effects row keyed (thread_id, step). INSERT ... ON CONFLICT DO NOTHING RETURNING 1: one caller gets a row and owns the step.
- **Marker**: a hidden HTML comment with thread and step in the comment body. Before posting, search the issue for it; if present, adopt instead of reposting.
- **Stale takeover**: ON CONFLICT DO UPDATE ... WHERE status='claimed' AND claimed_at < now() - interval. A done row is never retaken; a fresh claim is not stolen.

```
INSERT INTO side_effects (thread_id, step, status) VALUES (%s,%s,'claimed')
ON CONFLICT (thread_id, step) DO UPDATE SET claimed_at = now()
  WHERE side_effects.status = 'claimed'
    AND side_effects.claimed_at < now() - make_interval(secs => %s)
RETURNING 1
```

What Postgres does internally

- Concurrent inserts hit the primary key's unique index. The first inserts (uncommitted); the others find the key held by an uncommitted transaction and **wait on it**.
- On commit, waiters re-check: key exists, DO NOTHING, no row returned. If the first rolls back, one waiter becomes the winner. Correct even if the winner dies.
- DO UPDATE locks the existing row (like SELECT FOR UPDATE), then re-evaluates the WHERE on the latest version under READ COMMITTED, so two takeovers cannot both win.
- It is a **lease**, not a held lock: the side effect happens outside the DB, so ownership must expire.

Why not the alternatives

Option	Why not here
SELECT FOR UPDATE then post	Holds a transaction open across a slow HTTP call
pg_advisory_lock	Session-scoped; leaks/loses with pooled connections. Good for migrations, not this.
SERIALIZABLE	Conflicts abort; retries everywhere
Redis SETNX	Another system; still needs expiry and fencing

Guarantees (be precise)

- **Strong**: one winner among concurrent callers (DB row lock).
- **Best effort**: marker check after a crash (depends on GitHub list-after-write).
- **Gap to volunteer**: a slow-but-alive owner whose lease looks stale. Close it with a **fencing token** (monotonic counter incremented on every takeover; the write path rejects older tokens).
- True exactly-once against an API with no idempotency key is impossible; the honest claim is at-least-once made idempotent.

7. Q&A with model answers

Q: Have you deployed this to AWS?

Yes, as a test environment. I provisioned an EKS cluster in Sydney, pushed a hardened image to ECR, and deployed it by digest as two replicas on two nodes, with probes, limits and a zero-unavailable rolling update. I rehearsed a failed rollout and rollback there. It was reached by port-forward only: no public ingress, no production traffic, and no real OAuth login because the callback is tied to my Vercel domain. Along the way I debugged two organization SCPs and a Free Plan instance restriction.

Q: Why EKS instead of staying on Render?

For this project Render is simpler and fine. I used EKS to learn Kubernetes operations, because many teams run it at scale. For a small team without a need for the Kubernetes ecosystem I would pick ECS/Fargate or keep Render.

Q: How do you roll out a change that could affect millions of endpoints?

Never all at once. Rings or canary (1%, 10%, 100%), automatic halt on health signals, a kill switch, and a rollback that is rehearsed. Image by digest so what I tested is what ships. Content/config changes get the same staging as code.

Q: Why a memory limit but no CPU limit?

Memory is incompressible, so a limit is an OOM safety net. CPU is compressible and a limit uses CFS throttling that adds tail latency even on an idle node. Requests handle scheduling.

Q: Why maxUnavailable 0?

Capacity must not drop during a rollout. New pods must pass readiness first. The cost is needing headroom for maxSurge, and a bad image simply stalls instead of causing an outage.

Q: Postgres goes down. What do the probes do?

Liveness checks only the process, so pods are not restarted pointlessly. Readiness decides whether to serve; if it checks the DB, all pods go unready together and the Service has no endpoints, which is correct if the app cannot work, but dangerous if some endpoints do not need the DB. So I keep liveness trivial and make readiness deliberate.

Q: Why deploy by digest?

Tags are mutable: the same tag can point to different bytes later. A digest is content-addressed, so the exact tested image is what runs, and it blocks tag-swap attacks.

Q: How do pods get AWS permissions without keys?

IRSA or EKS Pod Identity: the pod's service account maps to an IAM role and receives short-lived credentials. IRSA needs an IAM OIDC provider, which my org's SCP denied, so Pod Identity is the design.

Q: How does CI authenticate to AWS?

GitHub OIDC: the workflow requests a token, assumes a role whose trust policy is scoped to my repo and branch, and gets short-lived credentials. No stored access keys.

Q: Two pods get the same resume request. What happens?

Without protection both run execute and post twice. With the claim row, Postgres makes exactly one INSERT succeed; the others skip. A marker in the comment makes a post-crash retry adopt the existing comment. The residual risk is a stale-but-alive owner, which a fencing token closes.

Q: What is the weakest part of your system?

The resume path: the checkpoint and the GitHub write are not atomic, and OAuth tokens are stored in plaintext including inside checkpoints. I know the fixes and have tested the first.

Q: A node drains and you lose all replicas. What was missing?

A PodDisruptionBudget and spread constraints across nodes/AZs. With two replicas on two nodes plus minAvailable 1, a drain would evict one at a time.

Q: How would you observe it?

Structured logs with correlation IDs, RED metrics, OpenTelemetry traces across graph nodes, alerts on SLOs, and rollout gating on error rate. Not built yet; designed.

Q: What would you change before production?

Pod Identity, Secrets Manager via CSI, ALB+ACM+WAF, PDB+HPA, NetworkPolicies, readOnlyRootFilesystem, image signing + admission control, a migration Job, split probes, canary rollout, private cluster endpoint, encrypted tokens.

Q: Where did you fail?

I recycled crash-looping pods before reading their logs and lost the evidence. Now I read logs --previous and describe first. I also wrote a script whose AWS calls defaulted to the wrong region; I made region an explicit input.

8. What is EKS, and why use it?

Kubernetes in one paragraph

Kubernetes is a declarative orchestrator. You describe desired state (3 replicas of this image, with these limits and probes) and controllers continuously reconcile reality to it: reschedule dead pods, roll out new versions, scale out. The key objects: Pod (one or more containers), ReplicaSet (keeps N pods), Deployment (rollout and rollback), Service (stable virtual IP and load balancing), Ingress (HTTP routing), ConfigMap/Secret, and Namespace (isolation).

EKS = AWS runs the control plane

Part	Who runs it	What it is
API server, etcd, scheduler, controller manager	AWS (multi-AZ, managed, about $0.10/h per cluster)	The brain: stores desired state, decides where pods go
Worker nodes (EC2: managed node groups, Karpenter, Fargate, or Auto Mode)	You (AWS manages AMI lifecycle for managed node groups)	Where your containers actually run (kubelet + container runtime)
Add-ons: vpc-cni, kube-proxy, coredns, metrics-server	Installed per cluster (I created these via the EKS API)	Pod networking, Service routing, cluster DNS, resource metrics
Your workloads, RBAC, network policies, upgrades of nodes	You	Shared responsibility model

Why use it (the honest case)

- **Standard, portable API and ecosystem**: Helm, Argo CD/Rollouts, Kyverno/OPA, Prometheus. Many companies, including security vendors operating at scale, run Kubernetes.
- **Declarative rollouts and rollbacks, self-healing, autoscaling, bin-packing** many services on shared nodes.
- **AWS integration**: IAM (Pod Identity/IRSA), ECR, ALB via the AWS Load Balancer Controller, VPC networking, CloudWatch/OpenTelemetry, KMS envelope encryption of secrets.
- **Offloads the hardest part** (highly available control plane, etcd backups, patching) so you operate the data plane.

Why NOT (say this; it shows judgment)

- Fixed cost (about $73/month per cluster before nodes, plus NAT/ALB) and real operational complexity (upgrades, add-ons, networking).
- For one small service, **ECS/Fargate, App Runner, Lambda, or Render** are simpler and cheaper. For ResolveFlow, EKS is overkill; I used it deliberately to learn operations.
- Choose Kubernetes when you have many services, need the ecosystem, multi-cloud portability, or platform-team scale.

Likely EKS questions and answers

Q: How does a pod call AWS APIs without keys?

The pod's service account is mapped to an IAM role (Pod Identity, or IRSA via an OIDC provider) and it receives short-lived credentials. Never node-wide credentials or stored keys.

Q: How does a Service get a public load balancer?

The AWS Load Balancer Controller watches Ingress/Service objects and provisions an ALB (or NLB), registering pod IPs as targets; TLS terminates with an ACM certificate.

Q: What happens if a node dies?

The control plane marks it NotReady, pods are rescheduled onto healthy nodes after a timeout, and a PodDisruptionBudget plus spread across AZs keeps capacity. Managed node groups replace the instance via the Auto Scaling group.

Q: How do you upgrade a cluster?

Upgrade the control plane first (one minor version at a time), then add-ons, then nodes via a rolling replacement of the node group, honoring PDBs. Test in a lower environment. Check API deprecations before upgrading.

Q: What did you run into in practice?

Org SCPs blocking region and IAM OIDC, a Free Plan limit on instance types that left the nodegroup creating forever, found by reading the Auto Scaling group activity log.

9. Can we use AWS as the vector database?

Short answer: yes. I verified that the S3 Vectors API and Bedrock are reachable in this account in ap-southeast-2. I have **not** built or benchmarked anything, so do not quote latency or cost as measured.

The options on AWS

Option	What it gives	Trade-off
S3 Vectors (vector buckets and indexes)	Cost-optimized vector storage inside S3 with a native query API and metadata filtering; no cluster to run	Dense similarity search only (no lexical/BM25); latency generally higher than a dedicated engine. Verify current limits
Bedrock Knowledge Bases	Managed pipeline: ingest, chunk, embed, store (S3 Vectors, OpenSearch, Aurora pgvector and others), Retrieve/RetrieveAndGenerate with citations	Less control over chunking and scoring; you inherit its defaults
OpenSearch (Service/Serverless)	Hybrid retrieval: BM25 lexical + vector, filters, aggregations	Most expensive and heaviest to operate
Postgres + pgvector	We already run Postgres; one fewer system; SQL filters; tsvector full-text search	Scale ceiling; competes with the app for DB resources

How I would add it to ResolveFlow (Phase 2 design)

- A provider-neutral interface: retrieve(query, k, filters) -> list of {id, text, source, score}. Implementations: PineconeRetriever (existing) and S3VectorsRetriever / BedrockKBRetriever. Select by feature flag; Pinecone stays the default until evals say otherwise.
- **Keep the same embeddings** (OpenAI text-embedding-3-small, 512 dims) when using S3 Vectors directly, so retrieval quality is comparable and no re-embedding is required. Bedrock KB would use its own embedding model.
- **Score calibration is the hidden trap.** independent_review gates on MIN_RELEVANCE_SCORE = 0.35, calibrated on Pinecone cosine similarity. S3 Vectors reports a distance; convert (for cosine, similarity = 1 - distance) and **re-calibrate the threshold** on a labeled set, or the groundedness gate silently changes behavior.
- Preserve metadata (id, source, repo, issue number) so citations still work and metadata filters (repo=...) are possible.
- Graceful errors: on provider failure, fall back or escalate to human; never post an ungrounded diagnosis.

Do we need OpenSearch/hybrid? (the sharp question)

Embeddings are fuzzy: exact tokens such as a stack-trace line, an exception class name, or '#4521' are poorly matched by semantic similarity. Before paying for OpenSearch, try cheaper steps: (1) extract issue IDs with a regex and use a **metadata filter / direct lookup**; (2) add **Postgres full-text search (tsvector)** for stack traces and fuse results with the vector hits using Reciprocal Rank Fusion; (3) only if recall@k still fails on exact-match cases, adopt OpenSearch hybrid.

How to decide with evidence (evaluation plan)

- Build a labeled set: real issues with the known-relevant snippets (the repo already has eval/real_test_cases.py to start from).
- Measure recall@k, citation relevance, p50/p95 retrieval latency, monthly cost, and a dedicated slice of exact-match queries (stack traces, IDs).
- Choose the default from the numbers; keep Pinecone available behind the flag.

Q: Why would you move from Pinecone to S3 Vectors?

Cost and consolidation: storage in S3 with pay-per-use queries and no capacity to manage, plus IAM-based access and Bedrock integration. I would only switch if recall and latency on my own eval set are acceptable, because it is dense-only and typically slower.

Q: What breaks when you swap vector stores?

Score semantics. My groundedness gate uses a similarity threshold calibrated on Pinecone. A different metric or scale changes which citations pass, so I must convert and recalibrate, and run the eval harness before flipping the flag.

Q: How would you handle stack traces and issue numbers?

Not with embeddings alone: regex extraction plus metadata filter for IDs, and lexical search (Postgres FTS, or OpenSearch if needed) fused with vector results via RRF.

10. Say / do not say, and cost cleanup

Say	Do not say
In my test environment...	In production...
I designed X; I have not deployed it	I deployed X (if you did not)
I hit this error; here is how I diagnosed it	It just worked
The trade-off is...	This is the best way
I don't know, here is how I would find out	Guessing confidently

Teardown (do not leave the cluster running)

```
eksctl delete cluster --name resolveflow-test --region ap-southeast-2 --profile resolveflow
# then verify: no clusters, no NAT gateways, no load balancers, ECR repo emptied/deleted
```

Approximate cost while running: EKS control plane about $0.10/h, NAT gateway about $0.06/h, 2 x t3.small about $0.05/h.

If you used an access key, deactivate and delete it and the IAM user when finished.